

Akademie MVP Features Document

Executive Summary

Akademie is a multi-tenant school management system specifically designed for the Cameroonian educational landscape. This document outlines the complete MVP feature set, current implementation status, and strategic roadmap to surpass existing solutions in the Cameroon market.

Target Market: Schools operating in Cameroon's dual educational system (Anglophone and Francophone) **Competitive Advantage:** Native support for both educational systems, modern UI/UX, multi-tenancy, and comprehensive feature set **MVP Timeline:** Q3 2024 - Q1 2025

Table of Contents

1. [Cameroonian Educational System Analysis](#)
2. [Competitive Landscape Analysis](#)
3. [Current Implementation Status](#)
4. [MVP Core Features](#)
5. [MVP Extended Features](#)
6. [Technical Architecture](#)
7. [Implementation Roadmap](#)
8. [Success Metrics](#)

Cameroonian Educational System Analysis

System Overview

Cameroon operates a unique **bilingual educational system** with two parallel subsystems:

1. **Anglophone System** (British model)
2. **Francophone System** (French model)

This duality stems from the country's colonial history and is enshrined in Law No. 98/004 of April 14, 1998. Both systems coexist, maintaining their specific methods of evaluation and certification.

Educational Structure

1. Secondary Education

General Secondary Education

Anglophone System (5+2 pattern)

- **Lower Secondary (First Cycle):** 5 years (Form 1-5)
 - Exit Exam: GCE O-Level (General Certificate of Education Ordinary Level)
- **Upper Secondary (Second Cycle):** 2 years (Lower & Upper Sixth)
 - Exit Exam: GCE A-Level (General Certificate of Education Advanced Level)

Francophone System (4+3 pattern)

- **Lower Secondary (Premier Cycle):** 4 years (6ème-3ème)
 - Exit Exam: BEPC (Brevet d'Études du Premier Cycle)
- **Upper Secondary (Second Cycle):** 3 years (Seconde-Terminale)
 - Exit Exams:
 - Probatoire (after Première)
 - Baccalauréat (after Terminale)

Technical/Vocational Secondary Education

Anglophone Technical

- **Lower Technical:** 5 years
 - Exit Exam: GCE O-Level (Technical)
- **Upper Technical:** 2 years
 - Exit Exam: GCE A-Level (Technical)

Francophone Technical

- **CAP (Certificat d'Aptitude Professionnelle):** 2-3 years
- **Brevet de Technicien (BT):** 2 years
- **Baccalauréat Technique:** 2 years

Streams and Specializations

Anglophone General Streams

- **Arts:** A1-A8 (Literature, Humanities, Social Sciences)
- **Science:** S1-S4 (Physics, Chemistry, Biology, Mathematics)

Francophone General Series

- **Literary:** A, B, C (Literature, Philosophy, Languages)
- **Scientific:** D, E (Mathematics, Physics, Chemistry, Biology)
- **Economic:** F, G (Economics, Management, Accounting)
- **Technical:** TI (Industrial), STT (Tertiary)

2. Higher Education (University)

LMD System (Licence-Master-Doctorat)

- Implemented in 2007 to harmonize both systems
- **Licence (Bachelor):** 3 years (L1-L3)
- **Master:** 2 years (M1-M2)
- **Doctorat (PhD):** 3+ years

Admission Requisites

- GCE A-Level passes in 2 subjects OR
- Baccalauréat certificate

Structure

- Faculties (Arts, Sciences, Medicine, Law, Economics, etc.)
- Technological Institutes
- Technical and Vocational Institutes

Key Administrative Bodies

1. **MINEDUB** (Ministry of Basic Education) - Primary education
2. **MINESEC** (Ministry of Secondary Education) - Secondary education
3. **MINESUP** (Ministry of Higher Education) - University education
4. **MINEFOP** (Ministry of Employment and Vocational Training) - Technical training
5. **MINEDUC** (Ministry of Education) - Overall coordination

Examination Bodies

1. **GCE Board** - Anglophone examinations (O-Level, A-Level)
2. **Office du Baccalauréat du Cameroun (OBC)** - Francophone examinations (Probatoire, Baccalauréat)
3. **Technical Examination Boards** - Technical/vocational certifications

Academic Calendar

- **Start:** September
- **End:** June
- **Duration:** 36 weeks of instruction
- **Terms:** Typically 3 terms per academic year

Pain Points in Current System

1. **Dual System Complexity:** Schools struggle to manage both systems
2. **Manual Processes:** Paper-based record-keeping, manual grade calculations
3. **Communication Gaps:** Limited parent-teacher communication
4. **Exam Management:** Complex tracking of exam registrations and results
5. **Fee Management:** Inconsistent fee collection and tracking
6. **Report Generation:** Time-consuming manual report card creation
7. **Data Silos:** Disconnected systems for different functions
8. **Limited Analytics:** No insights into student performance trends

Competitive Landscape Analysis

Existing School Management Systems in Cameroon

1. ProsoftAfrica School Management Software

Features:

- Student registration and management
- Grade management and report cards
- Position/ranking calculation
- Fee management
- Attendance tracking
- Administrative document generation

Strengths:

- Local presence in Cameroon
- Report card generation
- Position calculation

Weaknesses:

- Legacy UI/UX
- Limited multi-tenancy
- No native dual system support
- Limited mobile accessibility
- No public website integration

2. eSchool-SMS

Features:

- Student registration
- Grade management
- Notification system (SMS, email)
- Discipline management
- Administrative document generation
- Digital report cards
- Honor roll generation

Strengths:

- Communication features (SMS, email)
- Digital report cards
- Honor roll system

Weaknesses:

- Generic system not tailored to Cameroon
- Limited dual system support
- No multi-tenancy
- Outdated interface

3. Galactis.Education

Features:

- School management software
- French-language focus
- Administrative automation

Strengths:

- French-language support
- Administrative automation

Weaknesses:

- Francophone-focused (limited Anglophone support)
- Limited feature set
- No multi-tenancy

Competitive Gaps and Opportunities

What Existing Systems Lack:

1. Native Dual System Support

- No system properly handles both Anglophone and Francophone systems
- Generic grade calculation doesn't account for different grading scales
- Limited exam-specific features (GCE vs Baccalauréat)

2. Modern Multi-Tenancy

- Most systems are single-school installations
- No subdomain-based routing
- Limited scalability

3. Public Website Integration

- No built-in website builder
- Schools need separate systems for public presence
- No unified branding

4. Modern UI/UX

- Legacy interfaces
- Poor mobile experience
- Limited accessibility

5. Comprehensive Analytics

- Basic reporting only
- No predictive analytics

- Limited performance insights

6. Communication Hub

- Fragmented communication tools
- No integrated parent portal
- Limited notification systems

Akademee's Competitive Advantages:

1. True Dual System Native Support

- Built from ground up for Cameroon's dual system
- System-specific navigation and features
- Accurate grade calculations per system
- Exam-specific workflows

2. Modern Multi-Tenant Architecture

- Subdomain-based routing
- Tenant isolation
- Scalable cloud infrastructure
- Single codebase, multiple schools

3. Integrated Public Website

- Built-in website builder
- Template-based customization
- Unified branding
- SEO-optimized

4. Modern UI/UX

- React-based modern interface
- Mobile-responsive
- Dark mode support
- Accessibility-first design

5. Comprehensive Feature Set

- All-in-one solution
- No need for multiple systems
- Integrated workflows
- Data consistency

6. Bilingual Support

- English/French interface
- Localized content
- Cultural relevance

Current Implementation Status

Completed Features (Backend)

Authentication & Authorization

- ☒ User registration with school creation
- ☒ Login with subdomain, email, password
- ☒ JWT-based authentication
- ☒ Role-based access control (ADMIN, TEACHER, STUDENT, GUARDIAN)
- ☒ Password hashing with bcrypt
- ☒ School verification by subdomain
- ☒ Current user endpoint with school data

School Management

- ☒ School registration
- ☒ Subdomain-based routing
- ☒ School information management
- ☒ Email verification system
- ☒ Verification token generation and validation

Onboarding & Website

- ☒ Multi-step onboarding wizard
- ☒ Logo upload
- ☒ Primary color selection
- ☒ Template selection (Modern, Classic, Minimal)
- ☒ Website content management (tagline, description, stats, values)
- ☒ Hero image upload
- ☒ About photos management

- 🔄 Educational system selection
- 🔄 Onboarding completion tracking
- 🔄 Website publishing status
- 🔄 Public website data endpoint

Student Management

- 🔄 Student creation with user account
- 🔄 Student listing with pagination
- 🔄 Student retrieval by ID
- 🔄 Student update
- 🔄 Student deletion
- 🔄 Student number generation (auto or manual)
- 🔄 Class assignment (className and classId)
- 🔄 Enrollment tracking
- 🔄 Fee status tracking
- 🔄 Student status management (active, inactive, graduated, etc.)
- 🔄 Date of birth and gender tracking
- 🔄 Contact information (email, phone)

Guardian Management

- 🔄 Guardian creation
- 🔄 Guardian listing
- 🔄 Guardian update
- 🔄 Guardian-student relationships

Database Schema

- 🔄 Schools table with educational systems (JSONB)
- 🔄 Users table with school isolation
- 🔄 Roles table
- 🔄 User roles junction table
- 🔄 Students table with user relationship
- 🔄 Guardians table
- 🔄 Classes table
- 🔄 Subjects table
- 🔄 Grades table
- 🔄 Attendance table
- 🔄 Payments table
- 🔄 Media table for file storage
- 🔄 Enrollments table
- 🔄 Academic years table
- 🔄 Website templates table

Middleware

- 🔄 Authentication middleware (JWT verification)
- 🔄 Tenant middleware (school isolation)
- 🔄 School resolver middleware (subdomain extraction)
- 🔄 Error handling middleware
- 🔄 Role-based access middleware
- 🔄 Request validation middleware

API Endpoints

- 🔄 `/api/auth/register` - School and admin registration
- 🔄 `/api/auth/login` - User authentication
- 🔄 `/api/auth/verify-school` - School verification
- 🔄 `/api/auth/me` - Current user data
- 🔄 `/api/schools/onboarding` - Onboarding data
- 🔄 `/api/schools/onboarding/media` - Media upload
- 🔄 `/api/students` - Student CRUD operations
- 🔄 `/api/guardians` - Guardian CRUD operations
- 🔄 `/api/website/public` - Public website data
- 🔄 `/api/config` - Application configuration

Completed Features (Frontend)

Authentication

- 🔄 Login page with subdomain, email, password
- 🔄 Registration page
- 🔄 School verification
- 🔄 Auth context for global state
- 🔄 Protected routes
- 🔄 Role guards

Layout & Navigation

- ☒ Dynamic sidebar based on role
- ☒ Educational system-based navigation filtering
- ☒ Collapsible sidebar
- ☒ Header with theme/language toggles
- ☒ Mobile-responsive navigation

Onboarding

- ☒ Multi-step onboarding wizard
- ☒ Logo upload with preview
- ☒ Primary color selection with presets
- ☒ Educational system selection (multi-select)
- ☒ Template selection with previews
- ☒ Content input (tagline, description, stats, values)
- ☒ Review step
- ☒ Publish step

Student Management

- ☒ Student list page
- ☒ Add student drawer/modal
- ☒ Student form with validation
- ☒ Student table with status badges
- ☒ Empty state with CTA
- ☒ Loading states
- ☒ Error handling

Public Website

- ☒ Landing page template
- ☒ Dynamic school branding
- ☒ Template-based rendering
- ☒ Hero section
- ☒ Stats section
- ☒ Values section

Internationalization

- ☒ English and French support
- ☒ Translation infrastructure
- ☒ Language switching

Theming

- ☒ Light/dark mode
- ☒ Primary color customization
- ☒ Theme context

API Layer

- ☒ Axios configuration
- ☒ API service modules
- ☒ Endpoint definitions
- ☒ Error handling

In Progress Features

Student Management (Frontend)

- ☒ Student profile page
- ☒ Student editing functionality
- ☒ Student deletion confirmation
- ☒ Search and filtering

Not Started Features (Backend)

Class Management

- ☒ Class CRUD operations
- ☒ Class-student relationships
- ☒ Class teacher assignment
- ☒ Class capacity management
- ☒ Academic year association

Subject Management

- ☒ Subject CRUD operations
- ☒ Subject-class assignments

- ☐ Subject teacher assignments
- ☐ Credit/weight management

Grade Management

- ☐ Grade entry per subject
- ☐ Grade calculation per system (Anglophone vs Francophone)
- ☐ Term-based grading
- ☐ Average calculation
- ☐ Grade letter conversion
- ☐ Position/ranking calculation
- ☐ Report card generation

Attendance Management

- ☐ Daily attendance marking
- ☐ Attendance reporting
- ☐ Attendance statistics
- ☐ Attendance notifications

Finance Management

- ☐ Fee structure definition
- ☐ Fee assignment per class
- ☐ Payment recording
- ☐ Payment tracking
- ☐ Fee status calculation
- ☐ Payment reminders
- ☐ Financial reports

Exam Management

- ☐ Exam registration (GCE, Baccalauréat, etc.)
- ☐ Exam fee management
- ☐ Exam center assignment
- ☐ Exam result recording
- ☐ Exam certificate generation

Academic Calendar

- ☐ Academic year management
- ☐ Term/session management
- ☐ Holiday management
- ☐ Event calendar
- ☐ Schedule management

Communication

- ☐ Internal messaging
- ☐ Email notifications
- ☐ SMS notifications
- ☐ Parent portal
- ☐ Announcement system

Reporting & Analytics

- ☐ Student performance reports
- ☐ Class performance reports
- ☐ Attendance reports
- ☐ Financial reports
- ☐ Custom report builder
- ☐ Data export (PDF, Excel)

System Administration

- ☐ User management (beyond students)
- ☐ Role assignment
- ☐ Permission management
- ☐ Audit logging
- ☐ System configuration
- ☐ Backup/restore

Not Started Features (Frontend)

Class Management

- ☐ Class list page
- ☐ Class creation drawer
- ☐ Class detail page

- [🔗 Class student assignment](#)

Subject Management

- [🔗 Subject list page](#)
- [🔗 Subject creation drawer](#)
- [🔗 Subject detail page](#)

Grade Management

- [🔗 Grade entry page](#)
- [🔗 Grade book view](#)
- [🔗 Report card preview](#)
- [🔗 Grade analytics](#)

Attendance Management

- [🔗 Attendance marking page](#)
- [🔗 Attendance calendar view](#)
- [🔗 Attendance reports](#)

Finance Management

- [🔗 Fee structure page](#)
- [🔗 Payment recording page](#)
- [🔗 Payment history](#)
- [🔗 Financial dashboard](#)

Exam Management

- [🔗 Exam registration page](#)
- [🔗 Exam result entry](#)
- [🔗 Exam certificates](#)

Communication

- [🔗 Messaging interface](#)
- [🔗 Notification center](#)
- [🔗 Parent portal](#)
- [🔗 Announcement board](#)

Reporting

- [🔗 Report builder](#)
- [🔗 Report templates](#)
- [🔗 Export functionality](#)

Settings

- [🔗 School settings page](#)
- [🔗 User management page](#)
- [🔗 System configuration](#)

MVP Core Features

Priority 1: Must-Have for MVP Launch

1. Complete Student Management

Status: 70% Complete

Remaining Tasks:

- Student profile page with full details
- Student editing functionality
- Student deletion with confirmation
- Search and filtering (by class, status, name)
- Bulk student import (CSV/Excel)
- Student photo upload
- Guardian association in student creation

Backend Requirements:

- Add photo_url field to students table (migration)
- Add bulk import endpoint
- Add photo upload endpoint

- Add search/filtering optimization

Frontend Requirements:

- Student profile page component
- Edit student drawer
- Delete confirmation modal
- Search bar and filters
- Bulk import UI
- Photo upload component

2. Class Management

Status: 0% Complete

Required Features:

- Class CRUD operations
- Class-student relationships
- Class teacher assignment
- Class capacity management
- Academic year association
- Class list with student count
- Class detail view with enrolled students

Backend Requirements:

```
-- Classes table already exists, need to enhance
ALTER TABLE classes ADD COLUMN IF NOT EXISTS capacity INTEGER;
ALTER TABLE classes ADD COLUMN IF NOT EXISTS class_teacher_id INTEGER REFERENCES users(user_id);
ALTER TABLE classes ADD COLUMN IF NOT EXISTS academic_year_id INTEGER REFERENCES academic_years(academic_year_id);
```

API Endpoints:

- POST /api/classes - Create class
- GET /api/classes - List classes with student count
- GET /api/classes/:id - Get class details with students
- PUT /api/classes/:id - Update class
- DELETE /api/classes/:id - Delete class
- POST /api/classes/:id/students - Add students to class
- DELETE /api/classes/:id/students/:studentId - Remove student from class

Frontend Requirements:

- Class list page
- Class creation drawer
- Class detail page
- Student assignment interface
- Teacher selection dropdown

3. Subject Management

Status: 0% Complete

Required Features:

- Subject CRUD operations
- Subject-class assignments
- Subject teacher assignments
- Credit/weight management
- Subject list with class assignments

Backend Requirements:

```
-- Subjects table already exists, need to enhance
ALTER TABLE subjects ADD COLUMN IF NOT EXISTS credits INTEGER DEFAULT 1;
ALTER TABLE subjects ADD COLUMN IF NOT EXISTS coefficient INTEGER DEFAULT 1;
```

API Endpoints:

- POST /api/subjects - Create subject
- GET /api/subjects - List subjects
- GET /api/subjects/:id - Get subject details
- PUT /api/subjects/:id - Update subject
- DELETE /api/subjects/:id - Delete subject
- POST /api/subjects/:id/classes - Assign subject to class
- POST /api/subjects/:id/teachers - Assign teacher to subject

Frontend Requirements:

- Subject list page

- Subject creation drawer
- Subject detail page
- Class assignment interface
- Teacher assignment interface

4. Grade Management

Status: 0% Complete

Required Features:

- Grade entry per subject per term
- Grade calculation per educational system
- Term-based grading (Term 1, 2, 3)
- Average calculation
- Grade letter conversion per system
- Position/ranking calculation
- Report card generation (PDF)

Educational System Specifics:

Anglophone System:

- Scale: A (75-100), B (70-74), C (60-69), D (50-59), E (45-49), F (0-44)
- Average calculation: Weighted average based on subject coefficients
- Position: Class ranking based on average

Francophone System:

- Scale: 16-20 (Très Bien), 14-15.9 (Bien), 12-13.9 (Assez Bien), 10-11.9 (Passable), 8-9.9 (Insuffisant), 0-7.9 (Faible)
- Average calculation: Weighted average based on subject coefficients
- Mention: Based on average (Très Bien, Bien, Assez Bien, Passable, etc.)

Backend Requirements:

```
-- Grades table already exists, need to enhance
ALTER TABLE grades ADD COLUMN IF NOT EXISTS term VARCHAR(20);
ALTER TABLE grades ADD COLUMN IF NOT EXISTS academic_year_id INTEGER REFERENCES academic_years(academic_year_id);
ALTER TABLE grades ADD COLUMN IF NOT EXISTS coefficient INTEGER DEFAULT 1;
ALTER TABLE grades ADD COLUMN IF NOT EXISTS credits INTEGER DEFAULT 1;
ALTER TABLE grades ADD COLUMN IF NOT EXISTS grade_letter VARCHAR(5);
ALTER TABLE grades ADD COLUMN IF NOT EXISTS position INTEGER;
ALTER TABLE grades ADD COLUMN IF NOT EXISTS class_average DECIMAL(5,2);
```

Grade Calculation Service:

```
// Anglophone grade calculation
function calculateAnglophoneGrade(score) {
  if (score >= 75) return { letter: 'A', points: 5 };
  if (score >= 70) return { letter: 'B', points: 4 };
  if (score >= 60) return { letter: 'C', points: 3 };
  if (score >= 50) return { letter: 'D', points: 2 };
  if (score >= 45) return { letter: 'E', points: 1 };
  return { letter: 'F', points: 0 };
}

// Francophone grade calculation
function calculateFrancophoneGrade(score) {
  if (score >= 16) return { mention: 'Très Bien', points: 5 };
  if (score >= 14) return { mention: 'Bien', points: 4 };
  if (score >= 12) return { mention: 'Assez Bien', points: 3 };
  if (score >= 10) return { mention: 'Passable', points: 2 };
  if (score >= 8) return { mention: 'Insuffisant', points: 1 };
  return { mention: 'Faible', points: 0 };
}
```

API Endpoints:

- POST /api/grades - Create/update grade
- GET /api/grades - List grades with filters
- GET /api/grades/student/:studentId - Get student grades
- GET /api/grades/class/:classId - Get class grades
- POST /api/grades/calculate - Calculate averages and positions
- GET /api/grades/report/:studentId - Generate report card (PDF)

Frontend Requirements:

- Grade entry page (by class and subject)
- Grade book view (matrix: students x subjects)
- Report card preview
- Grade analytics dashboard
- Bulk grade entry

5. Attendance Management

Status: 0% Complete

Required Features:

- Daily attendance marking
- Attendance by class
- Attendance status (PRESENT, ABSENT, LATE, EXCUSED)
- Attendance reporting
- Attendance statistics
- Attendance notifications to parents

Backend Requirements:

```
-- Attendance table already exists, need to enhance
ALTER TABLE attendance ADD COLUMN IF NOT EXISTS marked_by INTEGER REFERENCES users(user_id);
ALTER TABLE attendance ADD COLUMN IF NOT EXISTS remarks TEXT;
```

API Endpoints:

- POST /api/attendance - Mark attendance
- GET /api/attendance - List attendance with filters
- GET /api/attendance/student/:studentId - Get student attendance
- GET /api/attendance/class/:classId - Get class attendance
- GET /api/attendance/statistics - Attendance statistics
- POST /api/attendance/bulk - Bulk attendance marking

Frontend Requirements:

- Attendance marking page (by class and date)
- Attendance calendar view
- Attendance report page
- Attendance statistics dashboard
- Quick attendance buttons (Present, Absent, Late)

6. Academic Year & Term Management

Status: 0% Complete

Required Features:

- Academic year CRUD
- Term/session management within academic year
- Current academic year setting
- Academic year activation/deactivation
- Term start/end dates

Backend Requirements:

```
-- Academic years table already exists
-- Need to add terms table
CREATE TABLE IF NOT EXISTS terms (
  term_id SERIAL PRIMARY KEY,
  school_id INTEGER REFERENCES schools(school_id),
  academic_year_id INTEGER REFERENCES academic_years(academic_year_id),
  name VARCHAR(50) NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  is_active BOOLEAN DEFAULT false,
  created_at TIMESTAMP DEFAULT NOW()
);
```

API Endpoints:

- POST /api/academics/years - Create academic year
- GET /api/academics/years - List academic years
- PUT /api/academics/years/:id - Update academic year
- POST /api/academics/terms - Create term
- GET /api/academics/terms - List terms
- PUT /api/academics/terms/:id - Update term
- POST /api/academics/years/:id/activate - Activate academic year

Frontend Requirements:

- Academic year management page
- Term management page
- Academic year selector in forms

7. Basic Finance Management

Status: 0% Complete

Required Features:

- Fee structure definition (per class)
- Fee assignment to students
- Payment recording
- Payment tracking
- Fee status calculation
- Basic financial reports

Backend Requirements:

```
-- Payments table already exists
-- Need to add fees table
CREATE TABLE IF NOT EXISTS fees (
  fee_id SERIAL PRIMARY KEY,
  school_id INTEGER REFERENCES schools(school_id),
  class_id INTEGER REFERENCES classes(class_id),
  name VARCHAR(200) NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  academic_year_id INTEGER REFERENCES academic_years(academic_year_id),
  due_date DATE,
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS student_fees (
  student_fee_id SERIAL PRIMARY KEY,
  student_id INTEGER REFERENCES students(student_id),
  fee_id INTEGER REFERENCES fees(fee_id),
  amount_due DECIMAL(10,2) NOT NULL,
  amount_paid DECIMAL(10,2) DEFAULT 0,
  status VARCHAR(20) DEFAULT 'pending',
  academic_year_id INTEGER REFERENCES academic_years(academic_year_id),
  created_at TIMESTAMP DEFAULT NOW()
);
```

API Endpoints:

- POST /api/finance/fees - Create fee structure
- GET /api/finance/fees - List fee structures
- POST /api/finance/fees/assign - Assign fees to students
- POST /api/payments - Record payment
- GET /api/payments - List payments
- GET /api/finance/student/:studentId - Get student fee status
- GET /api/finance/reports - Financial reports

Frontend Requirements:

- Fee structure page
- Fee assignment page
- Payment recording page
- Student fee status page
- Financial dashboard

Priority 2: Important for MVP

8. User Management (Teachers & Staff)

Status: 0% Complete

Required Features:

- Teacher/staff CRUD operations
- Role assignment
- Subject assignment to teachers
- Class assignment to teachers

- Teacher profile management

Backend Requirements:

```
-- Users table already exists
-- Need to add teacher-specific fields
ALTER TABLE users ADD COLUMN IF NOT EXISTS employee_number VARCHAR(50);
ALTER TABLE users ADD COLUMN IF NOT EXISTS date_of_hired DATE;
ALTER TABLE users ADD COLUMN IF NOT EXISTS qualification TEXT;
```

API Endpoints:

- POST /api/users - Create user (teacher/staff)
- GET /api/users - List users with role filter
- GET /api/users/:id - Get user details
- PUT /api/users/:id - Update user
- DELETE /api/users/:id - Delete user
- POST /api/users/:id/roles - Assign roles
- POST /api/users/:id/subjects - Assign subjects
- POST /api/users/:id/classes - Assign classes

Frontend Requirements:

- Teacher list page
- Teacher creation drawer
- Teacher profile page
- Role assignment interface
- Subject/class assignment interface

9. Report Card Generation

Status: 0% Complete

Required Features:

- PDF report card generation
- System-specific report card templates
- Student photo on report card
- School branding on report card
- Grade summary
- Position/ranking
- Teacher remarks
- Principal signature

Backend Requirements:

- PDF generation library (PDFKit or similar)
- Report card templates per educational system
- Digital signature support

API Endpoints:

- GET /api/reports/report-card/:studentId - Generate report card (PDF)
- GET /api/reports/transcript/:studentId - Generate transcript (PDF)

Frontend Requirements:

- Report card preview
- Download button
- Email report card option

10. Basic Analytics Dashboard

Status: 0% Complete

Required Features:

- Student enrollment statistics
- Attendance statistics
- Grade distribution
- Fee collection status
- Teacher workload
- Quick stats cards
- Basic charts

Backend Requirements:

- Statistics aggregation endpoints
- Data caching for performance

API Endpoints:

- GET /api/analytics/dashboard - Dashboard statistics
- GET /api/analytics/enrollment - Enrollment statistics

- GET /api/analytics/attendance - Attendance statistics
- GET /api/analytics/grades - Grade statistics
- GET /api/analytics/finance - Financial statistics

Frontend Requirements:

- Dashboard page with stats cards
 - Charts (using Chart.js or similar)
 - Date range filters
 - Export functionality
-

MVP Extended Features (Post-MVP)

Priority 3: Nice to Have

11. Exam Management

- Exam registration (GCE, Baccalauréat, etc.)
- Exam fee management
- Exam center assignment
- Exam result recording
- Exam certificate generation
- Exam statistics

12. Communication System

- Internal messaging
- Email notifications
- SMS notifications
- Parent portal
- Announcement system
- Push notifications

13. Advanced Reporting

- Custom report builder
- Scheduled reports
- Advanced analytics
- Predictive analytics
- Data export (PDF, Excel, CSV)
- Report templates

14. Library Management

- Book catalog
- Book borrowing
- Book returns
- Fine management
- Library statistics

15. Transport Management

- Bus route management
- Bus assignment
- Fee calculation
- Route optimization

16. Hostel/Dormitory Management

- Room allocation
- Room capacity
- Fee management
- Maintenance requests

17. Inventory Management

- Asset tracking
- Consumables management
- Purchase orders
- Vendor management

18. Timetable/Schedule Management

- Class schedule creation
- Teacher schedule
- Room allocation

- Conflict detection
- Schedule sharing

19. Parent Portal

- Student progress viewing
- Fee payment
- Communication with teachers
- Attendance viewing
- Report card access

20. Mobile App

- Student app
- Teacher app
- Parent app
- Push notifications
- Offline support

Technical Architecture

Backend Architecture

Technology Stack

- **Runtime:** Node.js 18+
- **Framework:** Express.js
- **Database:** PostgreSQL (Supabase)
- **ORM:** Direct SQL using postgres driver
- **Authentication:** JWT
- **File Storage:** Supabase Storage
- **Email:** Nodemailer

Architecture Pattern

- Service-Oriented Architecture
- Middleware-based request processing
- Tenant isolation at database level
- RESTful API design

Key Design Principles

1. **Multi-tenancy First:** All queries scoped by school_id
2. **Educational System Aware:** Features adapt to Anglophone/Francophone systems
3. **API-First:** Clean RESTful API design
4. **Scalability:** Horizontal scaling ready
5. **Security:** JWT auth, role-based access, input validation

Frontend Architecture

Technology Stack

- **Framework:** React 18
- **Build Tool:** Vite
- **Language:** JavaScript/JSX
- **Styling:** Tailwind CSS
- **State Management:** React Context API
- **Routing:** React Router
- **HTTP Client:** Axios
- **i18n:** react-i18next

Architecture Pattern

- Feature-based organization
- Component composition
- Custom hooks for logic reuse
- Service layer for API calls
- Context for global state

Key Design Principles

1. **Component Reusability:** Shared component library
2. **Responsive Design:** Mobile-first approach
3. **Accessibility:** WCAG AA compliance
4. **Performance:** Code splitting, lazy loading

5. **Developer Experience:** Clear structure, TypeScript-ready

Database Schema

Core Tables

- schools (with educational_systems JSONB)
- users (with school isolation)
- roles
- user_roles
- students
- guardians
- classes
- subjects
- grades
- attendance
- payments
- fees
- student_fees
- academic_years
- terms
- media
- enrollments

Key Relationships

- schools → users (one-to-many)
- users → students (one-to-one)
- students → guardians (many-to-many)
- classes → students (many-to-many via enrollments)
- classes → subjects (many-to-many)
- subjects → grades (one-to-many)
- students → grades (one-to-many)
- students → attendance (one-to-many)
- students → payments (one-to-many)

Implementation Roadmap

Phase 1: Core Student & Class Management (4 weeks)

Status: Starting

Week 1-2: Complete Student Management

- Student profile page
- Student editing
- Student deletion
- Search and filtering
- Bulk import
- Photo upload

Week 3-4: Class Management

- Class CRUD
- Class-student relationships
- Class teacher assignment
- Class detail page
- Student assignment interface

Phase 2: Academic Structure (3 weeks)

Week 5-6: Subject Management

- Subject CRUD
- Subject-class assignments
- Subject teacher assignments
- Credit/weight management

Week 7: Academic Year & Terms

- Academic year management
- Term management
- Current year activation

Phase 3: Grading System (4 weeks)

Week 8-9: Grade Entry

- Grade entry interface

- Grade book view
- Bulk grade entry
- Grade validation

Week 10-11: Grade Calculation & Reports

- Anglophone grade calculation
- Francophone grade calculation
- Position/ranking
- Report card generation (PDF)

Phase 4: Attendance & Finance (3 weeks)

Week 12: Attendance Management

- Daily attendance marking
- Attendance reporting -Attendance statistics

Week 13-14: Finance Management

- Fee structure
- Payment recording
- Fee status tracking
- Financial reports

Phase 5: User Management & Analytics (2 weeks)

Week 15: User Management

- Teacher/staff CRUD
- Role assignment
- Subject/class assignment

Week 16: Analytics Dashboard

- Dashboard statistics
- Charts and visualizations
- Quick stats

Phase 6: Testing & Polish (2 weeks)

Week 17: Testing

- End-to-end testing
- Bug fixes
- Performance optimization

Week 18: Documentation & Launch

- API documentation
- User documentation
- MVP launch preparation

Success Metrics

Technical Metrics

- **API Response Time:** < 200ms (p95)
- **Page Load Time:** < 2s
- **Uptime:** 99.5%
- **Database Query Time:** < 100ms (p95)

User Metrics

- **Onboarding Completion Rate:** > 80%
- **Daily Active Users:** Target 50 schools by end of Q1 2025
- **Feature Adoption:** > 70% of core features used weekly
- **User Satisfaction:** NPS > 50

Business Metrics

- **School Onboarding:** 10 schools/month by Q2 2025
- **Revenue:** Target 50M XAF/month by Q2 2025
- **Churn Rate:** < 5% monthly
- **Customer Acquisition Cost:** < 50K XAF/school

Competitive Metrics

- **Feature Parity:** Match or exceed top 3 competitors
 - **Unique Features:** 5+ features not available in competitors
 - **User Experience:** Superior UI/UX compared to competitors
 - **Performance:** Faster than competitors in key operations
-

Conclusion

This MVP document provides a comprehensive roadmap for building Akademee into the leading school management system in Cameroon. By focusing on the unique needs of the Cameroonian educational system—specifically the dual Anglophone/Francophone structure—Akademee will differentiate itself from generic solutions and existing competitors.

The phased implementation approach ensures that core functionality is delivered first, with extended features added based on user feedback and market demand. The technical architecture is designed for scalability, allowing the system to grow from MVP to a full-featured platform.

Key Success Factors:

1. **True Dual System Support:** Native implementation of both educational systems
2. **Modern UX:** Superior user experience compared to legacy systems
3. **Multi-Tenancy:** Scalable architecture for growth
4. **Comprehensive Features:** All-in-one solution reducing need for multiple systems
5. **Local Relevance:** Built specifically for Cameroon's educational landscape

By following this roadmap and maintaining focus on the unique value proposition, Akademee is positioned to become the default school management platform in Cameroon and potentially expand to other African countries with similar dual educational systems.

Document Version: 1.0 **Last Updated:** June 30, 2026 **Next Review:** August 30, 2026